

Multi-Item Capacitated Lot Sizing with Batch Production (MICLSP)

MIPster Test Suite — Problem Description & Formulation

All three variable types · Big-M constraints · Parameterisable difficulty

1 Motivation and Problem Class

The **Multi-Item Capacitated Lot Sizing Problem with Batch Production** (MICLSP) is a classical production-planning problem that arises whenever a shared machine must produce several items in discrete batches over a finite planning horizon.

It is an excellent benchmark for a mixed-integer programming solver because it *naturally* contains all three variable types that challenge a B&B solver, and the Big-M linking constraint appears organically from the problem structure rather than as a modelling artefact:

- **Binary variables** y_{it} : whether a costly machine setup is performed for item i in period t .
- **General-integer variables** n_{it} : the number of production batches of item i in period t (each batch contains a fixed number of units B_i).
- **Continuous variables** s_{it} : the inventory of item i held at the end of period t (fractional storage units arise from rounding and cost interpolation).
- **Big-M linking constraint**: a batch can only be produced if the machine was set up; the tightest valid coefficient $M_{it} = \lfloor C_t/B_i \rfloor$ comes directly from the machine capacity in period t .

The LP relaxation typically yields a fractional mix of setup and batch variables, so instances are rarely solved at the root node, making MICLSP a reliable stress test for the branching and cutting machinery of a MIP solver.

2 Problem Data

Let $\mathcal{I} = \{0, \dots, I-1\}$ be the set of items and $\mathcal{T} = \{0, \dots, T-1\}$ the set of time periods.

Symbol	Domain	Meaning
B_i	$\mathbb{Z}_{>0}$	Batch size of item i (units per batch)
d_{it}	$\mathbb{Z}_{\geq 0}$	Demand for item i in period t
f_i	$\mathbb{R}_{>0}$	Fixed setup cost for item i
h_i	$\mathbb{R}_{>0}$	Holding cost per unit of item i per period
C_t	$\mathbb{Z}_{>0}$	Machine capacity (units) in period t
M_{it}	$\mathbb{Z}_{\geq 0}$	Big-M coefficient: $M_{it} = \lfloor C_t/B_i \rfloor$

3 Decision Variables

Variable	Type	Meaning
y_{it}	Binary ($\{0, 1\}$)	1 if item i is set up in period t
n_{it}	Integer ($\mathbb{Z}_{\geq 0}$)	Number of batches of item i produced in period t
s_{it}	Continuous ($\mathbb{R}_{\geq 0}$)	Inventory of item i at the end of period t

The convention $s_{i,-1} = 0$ (no initial stock) is used throughout.

4 Mathematical Formulation

$$\min \sum_{i \in \mathcal{I}} \sum_{t \in \mathcal{T}} (f_i y_{it} + h_i s_{it}) \quad (1)$$

subject to:

$$s_{i,t-1} + B_i n_{it} - s_{it} = d_{it} \quad \forall i \in \mathcal{I}, t \in \mathcal{T} \quad (\text{inventory balance}) \quad (2)$$

$$\sum_{i \in \mathcal{I}} B_i n_{it} \leq C_t \quad \forall t \in \mathcal{T} \quad (\text{machine capacity}) \quad (3)$$

$$n_{it} \leq M_{it} y_{it} \quad \forall i \in \mathcal{I}, t \in \mathcal{T} \quad (\text{Big-M setup link}) \quad (4)$$

$$y_{it} \in \{0, 1\} \quad \forall i \in \mathcal{I}, t \in \mathcal{T} \quad (5)$$

$$n_{it} \in \mathbb{Z}_{\geq 0} \quad \forall i \in \mathcal{I}, t \in \mathcal{T} \quad (6)$$

$$s_{it} \geq 0 \quad \forall i \in \mathcal{I}, t \in \mathcal{T} \quad (7)$$

Sizes. An instance with I items and T periods has $I \cdot T$ binary variables, $I \cdot T$ general-integer variables, and $I \cdot T$ continuous variables, for a total of $3 \cdot I \cdot T$ variables and $I \cdot T + T + I \cdot T = T(2I+1)$ constraints (excluding variable bounds).

5 The Big-M Linking Constraint

Constraint (4) is the classic *Big-M* formulation for linking a semi-continuous variable to a binary indicator. Here, $M_{it} = \lfloor C_t / B_i \rfloor$ is the **tightest possible** coefficient: it equals the maximum number of batches of item i that could physically be produced in period t given the machine capacity C_t . Using this tight value (rather than an arbitrary large constant) strengthens the LP relaxation and reduces the integrality gap.

If $y_{it} = 0$, then $n_{it} \leq 0$, so $n_{it} = 0$ (no production without setup). If $y_{it} = 1$, the constraint $n_{it} \leq M_{it}$ is redundant, since the capacity constraint (3) already limits the total batches across all items.

Why Big-M is interesting here. The LP relaxation of (4) sets $n_{it} \leq M_{it} \hat{y}_{it}$ where $\hat{y}_{it} \in [0, 1]$. A fractional setup $\hat{y}_{it} = 0.4$ allows fractional batch counts up to $0.4 M_{it}$, which combined with fractional inventory $\hat{s}_{it}$ gives the LP solver large freedom to reduce setup costs while satisfying demands. This fractional exploitation is precisely what the B&B solver must overcome by branching on y_{it} .

6 LP Relaxation and Integrality Gap

The LP relaxation is obtained by replacing $y_{it} \in \{0, 1\}$ with $0 \leq y_{it} \leq 1$ and $n_{it} \in \mathbb{Z}_{\geq 0}$ with $n_{it} \geq 0$. Let z_{LP}^* and z_{MIP}^* denote the optimal values. Since this is a minimisation problem, $z_{\text{LP}}^* \leq z_{\text{MIP}}^*$.

The **relative integrality gap** is defined as

$$\Delta\% = 100 \cdot \frac{z_{\text{MIP}}^* - z_{\text{LP}}^*}{\max(1, z_{\text{LP}}^*)}.$$

Instances selected as test fixtures satisfy $\Delta\% > 0$, confirming that the root-node LP bound is strictly weaker than the MIP optimum and that B&B (with cuts and heuristics) is required to close the gap.

7 Random Instance Generation

Instances are generated deterministically from a seed using a linear congruential generator with the following distributions:

Parameter	Distribution	Range	Interpretation
B_i	Uniform integer	$[5, 30]$	Batch size
d_{it}	Uniform integer	$[1, 20]$	Period demand
f_i	Uniform integer	$[100, 500]$	Setup cost
h_i	Uniform integer	$[1, 10]$	Holding cost
C_t	Derived	$\lceil \rho \cdot \sum_i d_{it} \rceil$	Machine capacity

The **capacity tightness ratio** $\rho = C_t / \sum_i d_{it}$ is the key difficulty knob. With $\rho > 1$ the machine can accommodate all current-period demand on average, but the discrete batch structure and setup sequencing force careful scheduling. Typical values used in the test suite:

ρ	Effect	Typical solve
≥ 2.0	Very loose; most instances solved at root	< 1 s, $\Delta\% \approx 0$
1.5	Moderate; some branching required	1–10 s
1.25–1.35	Tight; clear LP-MIP gap, hundreds of nodes	10 s–2 min
1.10–1.20	Very tight; thousands of nodes	1–5 min
< 1.0	Often infeasible	—

For the test suite we target the $\rho = 1.25$ – 1.35 band: instances solve in 10 s–2 min with a non-trivial LP-MIP gap, exercising branching, cuts (MIR, GMI), and FeasibilityJump.

8 Illustrative Example ($I = 2$, $T = 3$)

Consider two items ($I = 2$) and three periods ($T = 3$) with the following data:

Item i	B_i	f_i	h_i	d_{it} for $t = 0, 1, 2$	t	C_t
0	10	200	2	8, 5, 12	0	30
1	15	150	3	6, 9, 4	1	30
					2	30

Variables: $y_{it} \in \{0, 1\}$, $n_{it} \in \mathbb{Z}_{\geq 0}$, $s_{it} \in \mathbb{R}_{\geq 0}$. Tight Big-M values: $M_{i0} = \lfloor 30/B_i \rfloor$, i.e. $M_{00} = 3$, $M_{10} = 2$ (similarly for other periods).

Optimal solution. In $t = 0$: set up both items ($y_{00} = y_{10} = 1$), produce 2 batches of item 0 (20 units) and 1 batch of item 1 (15 units), using capacity $10 \cdot 2 + 15 \cdot 1 = 35 > 30$. A feasible schedule might produce item 0 in $t = 0, 2$ and item 1 in $t = 0, 1$:

	$t = 0$			$t = 1$			$t = 2$		
	y	n	s	y	n	s	y	n	s
Item 0	1	2	12	0	0	7	1	1	5
Item 1	1	1	9	1	1	1	0	0	0

This example illustrates how the three variable types interact: the binary y triggers a setup cost, the integer n determines batch production, and the continuous s accumulates the remaining inventory.

9 Solution Verification in the Test Suite

After solving, the test binary verifies the following for every saved incumbent solution:

1. **Inventory balance** (equality constraint): for each (i, t) , $|s_{i,t-1} + B_i n_{it} - s_{it} - d_{it}| \leq \varepsilon$.
2. **Machine capacity** (inequality): $\sum_i B_i n_{it} \leq C_t + \varepsilon$.
3. **Big-M / no-production-without-setup**: if $n_{it} > 0$ then $y_{it} = 1$.
4. **Non-negativity**: $s_{it} \geq -\varepsilon$, $n_{it} \geq -\varepsilon$.
5. **Integrality**: n_{it} is within ε of an integer; $y_{it} \in \{0, 1\}$ within ε .
6. **Objective match**: the reported objective matches $\sum_{i,t} (f_i y_{it} + h_i s_{it})$ within ε .
7. **Known optimal**: the best solution objective matches the pre-computed optimal value stored in `miclsp_fixtures.h`.

Here $\varepsilon = 10^{-4}$ (the solver primal feasibility tolerance).

10 Test Fixture Instances

The table below lists all instances stored in `miclsp_fixtures.h`. Column “gap” is the LP–MIP integrality gap $\Delta\%$; “B&B” indicates whether at least one branch was performed (gap > 0).

Instance tag	I	T	ρ	Vars	Opt	LP bound	Gap
<i>(generated by <code>gen_miclsp_fixtures.py</code>)</i>							

The fixture header is regenerated by running:

```
cd Cbc/test
python3 gen_miclsp_fixtures.py > miclsp_fixtures.h
```

References

- [1] W. W. Trigeiro, L. J. Thomas, J. O. McClain, “Capacitated lot sizing with setup times,” *Management Science* 35(3):353–366, 1989.
- [2] Y. Pochet, L. A. Wolsey, *Production Planning by Mixed Integer Programming*, Springer, 2006.
- [3] C. P. M. van Hoesel, A. P. M. Wagelmans, “An $O(T^3)$ algorithm for the economic lot-sizing problem with constant capacities,” *Management Science* 42(1):142–150, 1996.